

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ
УНИВЕРСИТЕТ им. В. Г. ШУХОВА» (БГТУ им. В.Г. Шухова)
Кафедра программного обеспечения
вычислительной техники и автоматизированных
систем

Лабораторная работа №2
по дисциплине: Архитектура вычислительных систем
тема: «**Структура команд процессора**»

Выполнил: студент группы ВТ-231
Масленников Д. А.
Проверили:
Осипов О.В.
Бондаренко Т.В.

Белгород 2025

Цель работы: изучить структуру команд процессора, научиться составлять машинный код простейших команд.

Примечание: на лекции было сказано о том, что при выполнении лабораторной работы можно использовать операционную систему линукс. Поэтому, так как я привык работать именно в ней, лабораторная будет выполнена с использованием возможностей этой системы — в частности компилятор `nasm`, а так же отладчик `gdb`

Задания для выполнения к работе

1. Ознакомиться с теоретическим материалом главы 2 учебника В.И. Юрова «Assembler» «Программно-аппаратная архитектура IA-32 процессоров Intel».
2. В соответствии с вариантом задания определить по символьному описанию команд их машинный код (для 5 команд), а также по машинному коду команд определить их символьное описание (для 2 машинных кодов).

Задание по варианту №14:

Исходный код на MASM32:

```
ADD AX, 8080
AND BL, AL
TEST BYTE PTR [EBX+0E2h], 9
MOV DX, [ECX*4+34h]
OR [EAX], DX
```

```
8B441F 11
8B5D 02
```

Для выполнения заданий, я использовал программу, написанную на nasm, чтобы анализировать ее.

```
section .text
```

```
    global _start
```

```
_start:
```

```
    ; Часть 1: Символьные команды → машинный код
```

```
    add ax, 8080          ; ADD AX, 8080
```

```
    and bl, al           ; AND BL, AL
```

```
    test byte [ebx + 0xE2], 9      ; TEST BYTE PTR [EBX + 0E2h], 9
```

```
    mov dx, [ecx*4 + 0x34]        ; MOV DX, [ECX*4 + 34h]
```

```
    or [eax], dx               ; OR [EAX], DX
```

```
    ; Часть 2: Машинные коды → символные команды (вставлены как байты)
```

```
    db 0x8B, 0x44, 0x1F, 0x11
```

```
    db 0x8B, 0x5D, 0x02
```

```
    ; === Завершение программы ===
```

```
    mov eax, 1
```

```
    xor ebx, ebx
```

```
    int 0x80
```

После этого выполняем следующую комбинацию команд:

```
nasm -f elf32 lab2.asm -o lab2.o
```

```
ld -m elf_i386 lab2.o -o lab2
```

```
objdump -d lab2
```

Результат выполнения:

```
objdump -d lab2

lab2:      формат файла elf32-i386

Дизассемблирование раздела .text:

08049000 <_start>:
 8049000:    66 05 90 1f          add     $0x1f90,%ax
 8049004:    20 c3               and     %al,%bl
 8049006:    f6 83 e2 00 00 00 09  testb   $0x9,0xe2(%ebx)
 804900d:    66 8b 14 8d 34 00 00  mov     0x34(,%ecx,4),%dx
 8049014:    00
 8049015:    66 09 10           or      %dx,(%eax)
 8049018:    8b 44 1f 11        mov     0x11(%edi,%ebx,1),%eax
 804901c:    8b 5d 02          mov     0x2(%ebp),%ebx
 804901f:    b8 01 00 00 00    mov     $0x1,%eax
 8049024:    31 db          xor     %ebx,%ebx
 8049026:    cd 80          int     $0x80
```

Команда

1: ADD AX, 8080

Команда складывает 16-битный регистр AX с числом 8080 (1F90h).

Префикс 66 говорит о 16-битном размере данных в 32-битной среде. Код операции 05h специальный для ADD AX, imm16. Непосредственный операнд 90h 1Fh или 0001111110010000b — число 8080. В архитектуре x86 используется разделение на 2 байта — младший 90h и старший 1Fh

Машинный код: 66 05 90 1F. Длина — 4 байта.

Пре-фикс	Код операции (КОП)						SIB			Смещение	Непосредственный операнд
	05h	d	w	mod	reg/КОП	r/m	scale	index	base	-	1F90h
66h	00000101	-	1	-	000	-	-	-	-	-	0001111110010000b

Команда 2: AND BL, AL

Побитовое И между 8-битными регистрами BL и AL.

Код операции для побитового И между двумя 8-битными регистрами кодируется как 20h. D = 0, так как направление данных идет от регистра в поле reg к регистру в поле r/m. W = 0, так как размер данных — 8 бит. Mod 11, так как нет обращения к памяти и оба операнда — регистры. Reg — код регистра источника = 000, а r/m — назначение, то есть 011. Итоговый вид Mod + reg + r/m = 11000011 = C3.

Машинный код: 20 C3. Длина — 2 байта.

Пре-фикс	Код операции (КОП)						SIB			Смещение	Непосредственный операнд
-	20h	d	w	mod	reg/КОП	r/m	scale	index	base	-	-
-	00100000b	0	0	11	000	011	-	-	-	-	-

Команда 3: TEST BYTE PTR [EBX + 0E2h], 9

Побитовое И между байтом в памяти по адресу и непосредственным значением 9 без сохранения результата.

КОП = F6h(для TEST с байтовыми операндами и памятью), /0(указывает на TEST в байте r/m), mod = 10 (32битное смещение), r/m = 011 (код регистра EBX). Смещение: E2 00 00 00 — задает 32-битный адресный сдвиг относительно EBX(хотя это и помещается в 8 бит, но компилятор мог выбрать 32-битное смещение(mod=10), что увеличило длину команды до 7 байт), операнд: 09.

Машинный код: F6 83 E2 00 00 00 09. Длина — 7 байт.

Пре-фикс	Код операции (КОП)						SIB			Смещение	Непосредственный операнд
-	F6h	d	w	mod	reg/КОП	r/m	scale	index	base	E2 00 00 00	09h
-	11110110b	-	0	10	000	011	-	-	-		-

Команда 4: MOV DX, [ECX*4 + 34h]

Пересылка слова из памяти по адресу [ECX*4 + 34h] в DX.

Так как операция 16 битная в 32-битной среде, то префикс 66h. КОП = 8Bh для MOV из памяти в регистр для 16/32 битных операций. D = 1, так как данные из памяти r/m в регистр dx(reg). W = 1, так как размер данных — 16 бит(слово, регистр dx). Mod = 00 — двоичное. Reg = dx, r/m = 100. scale = 10(двоичная 4, указывает на масштабирование). Index = 001(ecx, индексный регистр). Base = 101(специальное значение, указывающее на отсутствие базового регистра при Mod=00 с 32-битным смещением). Смещение — 34 00 00 00, расширено до 32 бит, что характерно для случаев, когда адрес явно задается с учетом индекса и смещения. Может быть выбором компилятора для единообразия или спецификой сборки

Машинный код: 66 8B 14 8D 34 00 00 00. Длина — 8 байт.

Пре-фикс	Код операции (КОП)						SIB			Смещение	Непосредственный операнд
66h	8Bh	d	w	mod	reg/КОП	r/m	scale	index	base	34 00 00 00	
-	10001011 b	1	1	00	010	100	10	001	101		

Команда 5: OR [EAX], DX

Побитовое ИЛИ слова по адресу [EAX] с регистром DX.

Префикс 66h, уже понятно для чего. D = 0, так как результат записывается в память. W = 1, так как размер 16 бит(слово). Mod = 00, прямое обращение к памяти через eax. Reg = 010 = dx, r/m = 000 = eax(указывает на базовую адрессацию памяти).

Машинный код: 66 09 10. Длина — 3 байта.

Пре-фикс	Код операции (КОП)						SIB			Смещение	Непосредственный операнд
66h	09h	d	w	mod	reg/КОП	r/m	scale	index	base		
	00001001 b	0	1	00	010	000					

Часть 1: Символьное → машинный код

1 ADD AX, 8080	66 05 90 1F
2 AND BL, AL	20 C3
3 TEST BYTE PTR [EBX + 0E2h], 9	F6 83 E2 00 00 00 09
4 MOV DX, [ECX*4 + 34h]	66 8B 14 8D 34 00 00 00
5 OR [EAX], DX	66 09 10

Машинный код 1: 8B441F 11

8B — MOV r32, r/m32.

44 → mod=01, reg=000 (EAX), r/m=100 → SIB.

1F → scale=00, index=011 (EBX), base=111 (EDI). Смещение: 11h.

Команда: MOV EAX, [EBX + EDI + 11h].

Машинный код 2: 8B5D 02

8B — MOV r32, r/m32.

5D → mod=01, reg=011 (EBX), r/m=101 → [EBP + disp8]. Смещение: 02h.

Команда: MOV EBX, [EBP + 2].

Часть 2: Машинный код → символьное

8B 44 1F 11	MOV EAX, [EBX + EDI + 11h]
8B 5D 02	MOV EBX, [EBP + 2]

Вывод:

В ходе выполнения лабораторной работы изучена структура машинных команд процессора IA-32. Для заданных символьных команд определены их машинные коды, а по заданным машинным кодам восстановлено символьное представление. Все результаты подтверждены с помощью ассемблера NASM и утилиты objdump.